Early-Exit Speculative Decoding for Hindi: A Comprehensive Final Report

Team: Transformers
KSPSVLN Siddardha Kumar Kavuri (2025201061)
Piyush Kumar Agarwal (2025201082)
Sankalp Chaturvedi (2025201083)
Aviral Tyagi (2025201086)

Abstract

Autoregressive decoding in large language models is inherently sequential, with each token requiring a full forward pass through all transformer layers. Early-Exit Speculative Decoding (EESD) addresses this by attaching lightweight exit heads at intermediate layers of the target model itself, eliminating the need for a separate draft model. We present a comprehensive evaluation of EESD applied to **Qwen2-1.5B** for **Hindi text generation**, comparing **12 decoding strategies** spanning fixed-depth exits, bottleneck heads, and adaptive selection policies (Thompson Sampling, UCB, Entropy, and weighted variants). On 101 Hindi prompts, BN-TrueExit achieves the highest acceptance rate ($\alpha = 70.9\%$), while Thomp-BN delivers the best wall-clock speedup (1.130×) and throughput (30.40 Tok/s), the only method to exceed the autoregressive baseline (26.58 Tok/s). A 500-sample robustness evaluation confirms that method rankings remain stable at scale, with Thomp-BN sustaining a speedup above 1.0× (1.041×). Morphological analysis by UPOS tag reveals that functional categories (ADP, AUX) are near-perfectly predicted by exit heads, while content-bearing categories (VERB, PROPN) remain challenging. Training diagnostics comparing Hook and Bottleneck heads show that the bottleneck architecture converges to lower loss and higher acceptance, explaining its superior inference-time performance. Our results demonstrate that EESD is a viable acceleration strategy for morphologically rich languages, with clear, quantified trade-offs between acceptance quality, latency, and throughput.

1 Introduction

Large Language Models (LLMs) have become the backbone of modern NLP, yet their autoregressive decoding paradigm remains a fundamental bottleneck: generating N tokens requires N sequential forward passes, each traversing every transformer layer. For a 28-layer model producing 100 tokens, this amounts to 2,800 layer evaluations with no parallelism across the token dimension.

Speculative decoding (Leviathan et al., 2023; Chen et al., 2023) alleviates this by using a lightweight *draft model* to propose multiple candidate tokens that the full *target model* verifies in a single batched pass. Accepted tokens bypass the sequential bottleneck, yielding wall-clock speedups without altering the output distribution. However, maintaining a separate draft model doubles parameter storage and complicates deployment.

Early-Exit Speculative Decoding (EESD) (Liu et al., 2024) resolves this by reusing the target model's own intermediate representations. Lightweight exit heads—comprising a LayerNorm and a linear projection—are attached at selected intermediate layers. During drafting, inference can stop at the exit layer rather than running all layers, providing both speed and distribution alignment with negligible additional parameters.

In this project, we implement EESD for **Hindi text generation** using **Qwen2-1.5B** (Yang et al., 2024) as the base model and conduct the most comprehensive evaluation of the approach to date. Hindi serves as an informative testbed because its rich morphology (inflected verbs, compound words, postpositions) poses a genuine challenge for draft-model accuracy.

Contributions.

1. A complete comparison of **12 decoding methods**, including fixed-depth exits

(Draft, Hook, TrueExit, BN-TrueExit), adaptive policies (Thompson, Thomp-BN, Entropy), bandit-based policies (UCB-BN, UCB-Hook), and weighted variants (WThomp-BN, WThomp-Hook).

2. Evaluation on 101 Hindi prompts and a larger **500-sample** robustness study, reporting acceptance rate (α), wall-clock speedup, and throughput (Tok/s).

3. **Morphological analysis** stratified by UPOS tag, revealing category-specific acceptance behaviour.

4. **Hook vs. Bottleneck training diagnostics**, connecting training dynamics to inference outcomes.

5. Depth analysis, K-ablation, and practical deployment recommendations.

Code Repository. Implementation, training scripts, and evaluation pipelines are available at: GitHub Repository: Speculative-Decoding-for-Hindi

Link to source code, models: Link

2 Background and Related Work

2.1 Autoregressive Decoding

Standard autoregressive (AR) generation produces one token at a time: $x_t \sim p_\theta(x_t \mid x_{<t})$. Each step requires a full forward pass through all L transformer layers, making inference time linear in both sequence length and model depth.

2.2 Speculative Decoding

Speculative decoding (Leviathan et al., 2023; Chen et al., 2023; Stern et al., 2018) introduces a draft-then-verify paradigm:

1. A lightweight draft model q_ϕ generates K candidate tokens autoregressively.

2. The full target model p_θ processes the prompt plus all K candidates in a single forward pass.

3. Tokens are accepted greedily until the first mismatch; the correct token at the mismatch position is sampled from p_θ.

If the draft model's acceptance rate is α, the expected speedup is proportional to $K \cdot \alpha$ when draft generation is much cheaper than target verification (Xia et al., 2024). Variants include Medusa (Cai et al., 2023), which adds multiple prediction heads, and Draft & Verify (Zhang et al., 2023), which achieves lossless acceleration via self-speculative decoding.

2.3 Early Exiting

Early exiting attaches classifiers at intermediate layers and routes easy inputs to shallow exits. BranchyNet (Teerapittayanon et al., 2016) pioneered this for CNNs; DeeBERT (Xin et al., 2020) and CALM (Schuster et al., 2022) adapted it to transformers. Bae et al. (Bae et al., 2023) propose synchronized parallel decoding for fast early exiting in autoregressive models.

2.4 EESD and Adaptive Policies

Liu et al. (Liu et al., 2024) combine early exiting with speculative decoding, attaching exit heads at intermediate layers of the target LLM. Key advantages include: no external draft model ($< 0.5\%$ added parameters), inherent distribution alignment from intermediate representations, and flexible depth selection. The original paper further introduces **Thompson Sampling** to dynamically select the optimal exit depth at each step, maximising expected speedup.

In our work, we extend this to include **bottleneck heads** (reduced-dimension projection before the vocabulary layer), **UCB-based policies** (Upper Confidence Bound exploration), **entropy-based gating**, and **weighted Thompson variants** that incorporate acceptance-rate history into the sampling distribution. Reference links for these adaptive controls: Thompson Sampling tutorial, UCB1 paper, and Entropy-based confidence gating (CALM).

3 Methodology

3.1 Architecture

Base Model. We use **Qwen2-1.5B** (Yang et al., 2024), a 28-layer transformer decoder with hidden dimension 1,536 and vocabulary size 151,936. The model is loaded in float16 precision with all base parameters

frozen throughout training.

Exit Heads. At each selected layer depth $d \in \{8, 16, 22\}$, we attach a lightweight exit head:

$$\hat{y}_d = \text{Linear}\big(\text{LayerNorm}(h_d)\big) \quad (1)$$

where $h_d \in \mathbb{R}^{B \times T \times 1536}$ is the hidden state after the d-th transformer layer, and the linear projection maps to the full vocabulary size.

Head Variants. We evaluate two head architectures:

- **Hook head**: standard LayerNorm + Linear projection directly to vocabulary size ($1536 \to 151936$).
- **Bottleneck (BN) head**: LayerNorm + Linear down-projection ($1536 \to 256$) + activation + Linear up-projection ($256 \to 151936$), reducing the parameter count and providing a compressed representation bottleneck.

3.2 Training Objective

We train exit heads using **soft KL-divergence distillation** (Hinton et al., 2015):

$$\mathcal{L} = \frac{1}{|\mathcal{D}|} \sum_{d \in \mathcal{D}} T^2 \cdot D_{\text{KL}}\big(\sigma(f/T) \parallel \sigma(\hat{f}_d/T)\big) \quad (2)$$

where $\mathcal{D} = \{8, 16, 22\}$, f and $\hat{f}_d$ are the full-model and exit-head logits, $T = 2.0$ is the distillation temperature, and σ denotes softmax.

3.3 Training Setup

- **Data:** 167K Hindi sentences from AI4Bharat IndicCorpv2 (Doddapaneni et al., 2021) (≈5M tokens).
- **Optimiser:** 8-bit AdamW via `bitsandbytes`.
- **Learning rate:** 10^{-4} with cosine annealing and 5% linear warmup.
- **Batch size:** 2 per GPU × 4 gradient accumulation (effective batch size 8).
- **Epochs:** 10.
- **Hardware:** NVIDIA H100 (training); A100 (inference).
- **Precision:** float16 base model, float32 loss computation.

3.4 Decoding Methods

We evaluate 12 decoding strategies, summarised below:

1. **AR**: Standard autoregressive baseline (no drafting).
2. **Draft**: Fixed-depth speculative decoding using the deepest exit head (layer 22).
3. **Hook**: Fixed-depth decoding using the standard Hook head.
4. **TrueExit**: Fixed-depth decoding with true early exit (forward pass stops at exit layer).
5. **BN-TrueExit**: True early exit using the Bottleneck head variant.
6. **Thompson**: Thompson Sampling over exit depths (standard Hook heads).
7. **Thomp-BN**: Thompson Sampling with Bottleneck heads.
8. **Entropy**: Entropy-based gating that selects the shallowest exit whose output entropy is below a learned threshold.
9. **UCB-BN**: UCB policy with Bottleneck heads, using reward weights $w_{\text{correctness}} = 0.92$ and $w_{\text{time}} = 0.08$ (as implemented in `src/evaluate_all.py`) with UCB exploration constant $c = 1.41$ (`src/model.py`).
10. **UCB-Hook**: UCB policy with Hook heads, with the same weights $w_{\text{correctness}} = 0.92$, $w_{\text{time}} = 0.08$, and $c = 1.41$.
11. **WThomp-BN**: Weighted Thompson Sampling with Bottleneck heads using $w_{\text{correctness}} = 0.92$ and $w_{\text{time}} = 0.08$ (`src/evaluate_all.py`).
12. **WThomp-Hook**: Weighted Thompson Sampling with Hook heads using $w_{\text{correctness}} = 0.92$ and $w_{\text{time}} = 0.08$.

3.5 Evaluation Metrics

- **Acceptance Rate (α)**: Fraction of draft tokens accepted by the full model during verification.
- **Speedup**: Ratio of AR wall-clock time to EESD wall-clock time. Values $> 1\times$ indicate faster-than-AR decoding.

- **Throughput (Tok/s)**: Tokens generated per second, capturing end-to-end efficiency.

4 Experimental Setup

4.1 Evaluation Data

101-Prompt Set. We curate 101 diverse Hindi prompts spanning Indian geography, culture, history, science, politics, sports, economy, and daily life (10–25 tokens each), designed to elicit varied morphological structures.

500-Sample Set. To assess robustness at larger scale, we evaluate all methods on 500 Hindi prompts drawn from a broader distribution, reporting acceptance and speedup separately.

Morphological Evaluation. For UPOS-stratified analysis, we use the test split of **XL-Sum Hindi** (Hasan et al., 2021), tagging generated tokens with a Hindi UPOS tagger and computing per-category acceptance rates.

4.2 Inference Protocol

All methods are evaluated with draft length $K = 3$ and `max_new_tokens`$= 100$ on identical hardware (NVIDIA A100). Wall-clock times are averaged over 3 runs to reduce variance.

5 Results

5.1 Main Results

Table 1 presents the complete results across all 12 methods on the 101-prompt evaluation set, alongside the 500-sample robustness columns.

Table 1: Summary of all 12 methods on 101 Hindi prompts (main) and 500 Hindi prompts (robustness). Best values (excluding AR) are **bolded**.

Method	α	Spd.	Tok/s	500 α	500 Spd.
AR	—	1.000	26.58	—	1.000
Draft	65.2%	0.707	19.09	64.4%	0.673
Hook	46.9%	0.552	14.89	45.8%	0.529
TrueExit	60.9%	0.736	20.11	59.7%	0.701
BN-TrueExit	**70.9%**	0.796	21.50	**68.8%**	0.751
Thompson	48.9%	1.054	28.29	48.4%	0.985
Thomp-BN	54.8%	**1.130**	**30.40**	54.5%	**1.041**
Entropy	56.5%	0.570	15.34	55.8%	0.532
UCB-BN	62.3%	0.919	24.81	60.7%	0.869
UCB-Hook	52.9%	0.578	15.57	52.0%	0.554
WThomp-BN	62.9%	0.876	24.13	60.7%	0.872
WThomp-Hook	53.4%	0.574	15.80	52.5%	0.559

Figures 1–4 visualise acceptance rate, speedup, throughput, and the acceptance–speedup trade-off across all 12 methods.

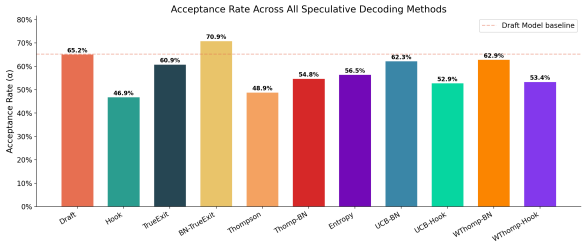

Figure 1: Acceptance rate (α). BN-TrueExit leads at 70.9%, followed by Draft (65.2%) and WThomp-BN (62.9%).

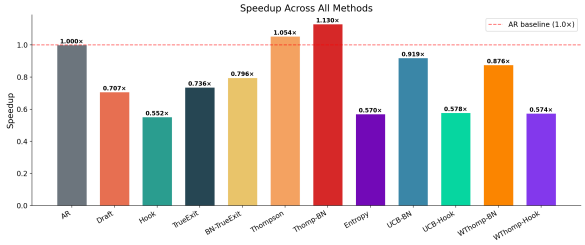

Figure 2: Wall-clock speedup. Only Thomp-BN (1.130×) and Thompson (1.054×) exceed 1.0×. Hook-based methods consistently trail Bottleneck counterparts.

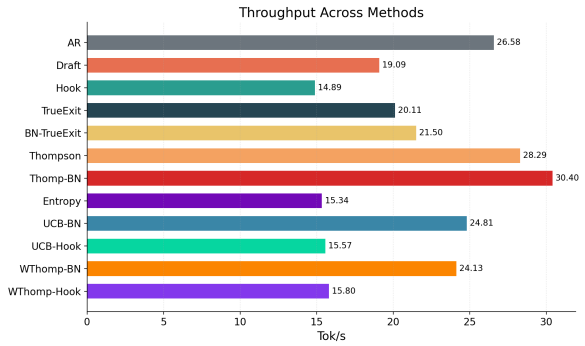

Figure 3: Throughput (Tok/s). Only Thompson (28.29) and Thomp-BN (30.40) have throughput above the AR baseline (26.58).

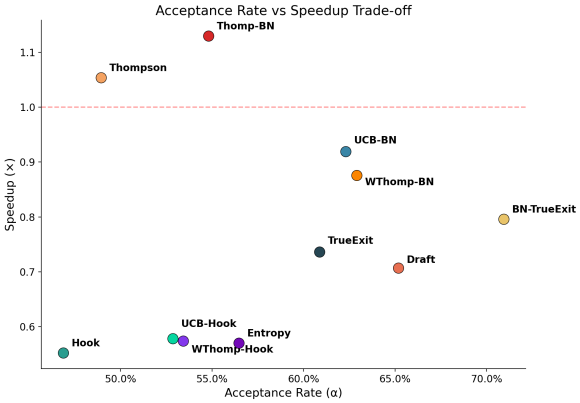

Figure 4: Acceptance vs. speedup trade-off. Thomp-BN leads on speed; BN-TrueExit leads on α. High-acceptance methods cluster upper-left; high-speed methods cluster right; Hook-based methods cluster lower-left.

Key observations.

- **Best acceptance**: BN-TrueExit achieves 70.9%, demonstrating that the bottleneck head with true early exit best approximates the full model distribution.
- **Best speedup**: Thomp-BN at 1.130× is the only method delivering meaningful wall-clock gains. Thompson (standard heads) also exceeds 1.0× at 1.054×.
- **Best throughput**: Thomp-BN at 30.40 Tok/s surpasses the AR baseline (26.58 Tok/s) by 14.4%.
- **Bottleneck advantage**: For every policy family, the BN variant outperforms its Hook counterpart on both speedup and throughput (e.g., Thomp-BN 1.130× vs. Thompson 1.054×; UCB-BN 0.919× vs. UCB-Hook 0.578×).
- **Acceptance–speed gap**: BN-TrueExit has the highest α but only 0.796× speedup, because fixed-depth strategies incur full verification overhead on every step. Adaptive policies sacrifice some acceptance for substantially better latency.
- **Trade-off clusters** (Figure 4): High-acceptance methods (BN-TrueExit, Draft, UCB-BN) occupy the upper-left; high-speed methods (Thomp-BN, Thompson) occupy the right; Hook-based methods cluster in the lower-left ($\alpha < 55\%$, speedup $< 0.6\times$).

5.2 Acceptance–Speedup Trade-off

The scatter in Figure 4 reveals three distinct clusters:

1. **High-acceptance, moderate-speed** (upper-left): BN-TrueExit, Draft, UCB-BN, WThomp-BN — these methods prioritise distribution fidelity.
2. **Moderate-acceptance, high-speed** (right): Thomp-BN, Thompson — these methods exploit adaptive depth selection for latency gains.
3. **Low-acceptance, low-speed** (lower-left): Hook, UCB-Hook, WThomp-Hook, Entropy — Hook-based methods without true early exit or adaptive policies.

5.3 Depth Analysis

Figures 5 and 6 examine how exit depth affects acceptance and how adaptive policies distribute their depth selections.

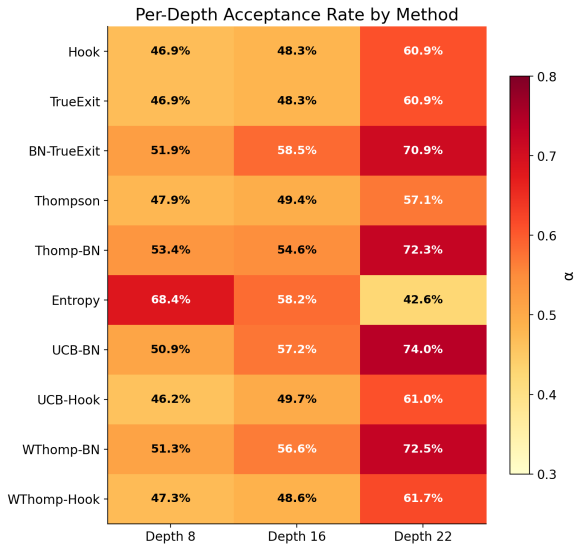

Figure 5: Per-depth acceptance heatmap. Deeper layers (22) consistently yield higher α; bottleneck variants show the steepest gradient.

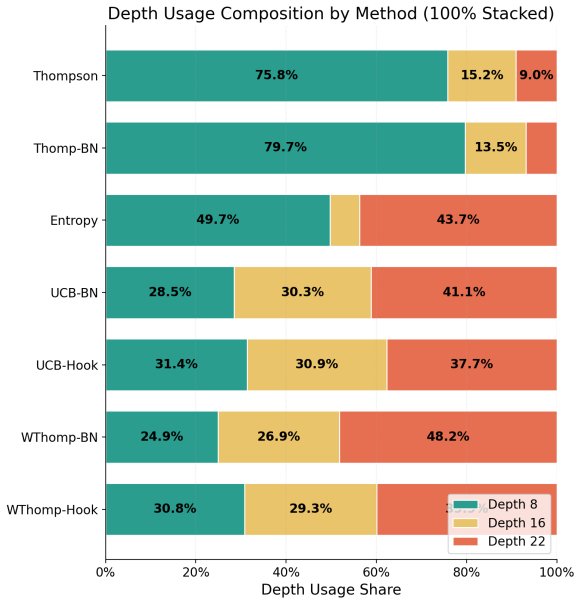

Figure 6: Adaptive depth usage. Thompson-family policies dynamically balance shallow (cheap) vs. deep (accurate) exits.

Interpretation. The heatmap confirms a monotonic relationship between exit depth and acceptance rate: layer 22 consistently achieves the highest α across all head variants. The depth-usage plot reveals that Thompson-family policies do not simply default to the deepest layer; instead, they dynamically balance exploration of shallower exits (which are cheaper per step) against the higher acceptance of deeper exits. This principled exploration–exploitation trade-off is the core mechanism behind their speedup advantage.

5.4 K-Ablation

Figure 7 presents the effect of draft length K on acceptance and speedup.

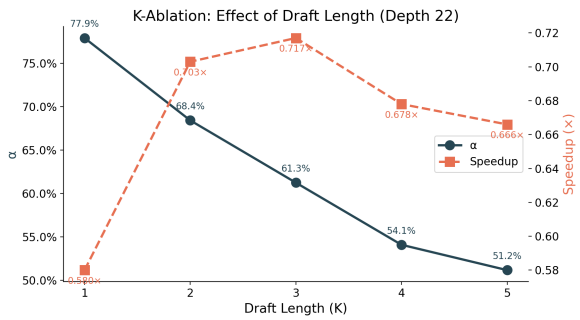

Figure 7: K-ablation. $K = 3$ offers the best balance; higher K yields diminishing returns as later draft positions have cumulatively lower acceptance.

The ablation confirms that $K = 3$ is near-optimal for our acceptance-rate regime. At $K = 1$, the verification overhead relative to a single drafted token is too high. At $K \geq 5$, later drafted tokens are increasingly likely to be rejected, which wastes computation. The expected number of accepted tokens per step, $\frac{\alpha(1-\alpha^K)}{1-\alpha}$, saturates quickly for $\alpha < 0.75$. We deliberately run this K-ablation with *fixed-depth TrueExit* at depth $d = 22$ to isolate the effect of K under a controlled setting: changing policy (e.g., Thompson/UCB) or depth simultaneously would confound attribution. Depth 22 was chosen because it is the strongest fixed-depth operating point in our earlier depth analysis, so the resulting K trend is both interpretable and practically relevant. This is a scoped ablation (not a full method$\times$depth$\times K$ factorial), chosen to provide a clean first-order characterization of K while keeping compute tractable.

5.5 Hook vs. TrueExit Timing

Figure 8 compares the per-depth wall-clock time of Hook and TrueExit methods.

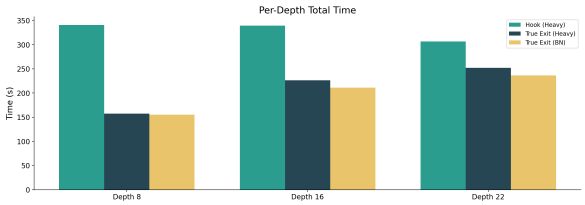

Figure 8: Hook vs. TrueExit timing. TrueExit saves computation by stopping at the exit layer; the gap is largest at shallow depths.

The timing plot directly explains the speedup gap between TrueExit/BN-TrueExit and their Hook counterparts: TrueExit avoids running the remaining $L - d$ layers during drafting, translating the acceptance-rate advantage into actual wall-clock savings.

5.6 500-Sample Robustness

To verify that our findings generalise beyond the 101-prompt evaluation set, we evaluate all methods on 500 Hindi prompts. Figure 9 and Figure 10 visualise the 500-sample results.

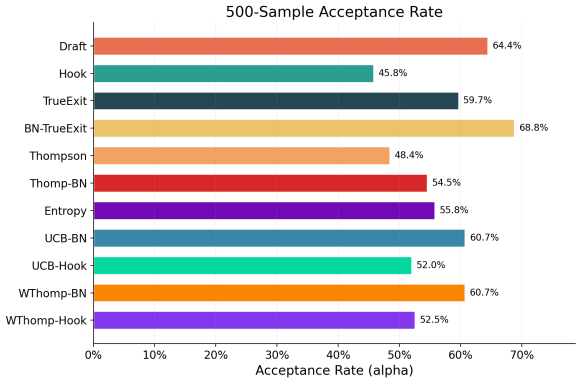

Figure 9: 500-sample acceptance. BN-TrueExit leads at 68.8%, followed by Draft (64.4%). Method ordering is fully preserved from the 101-prompt evaluation.

Figure 10: 500-sample speedup. Thomp-BN remains above 1.0× (1.041×). Thompson drops just below unity (0.985×), suggesting a less robust margin.

Robustness analysis. Comparing the 101-prompt and 500-prompt columns in Table 1:

- Acceptance drops are modest across all methods: BN-TrueExit drops from 70.9% to 68.8% (−2.1 pp), Thomp-BN from 54.8% to 54.5% (−0.3 pp).
- Speedup reductions are similarly contained: Thomp-BN drops from 1.130× to 1.041× but remains above unity.
- Thompson (standard) drops from 1.054× to 0.985×, falling just below the AR baseline at the larger scale, suggesting its margin is less robust.
- Method *ordering* is entirely preserved: BN-TrueExit > Draft > WThomp-BN ≈ UCB-BN for acceptance; Thomp-BN > Thompson > UCB-BN for speedup.

This stability confirms that our 101-prompt conclusions are not artefacts of a small evaluation set.

5.7 Morphological (UPOS) Analysis

Table 2 and Figure 11 present acceptance rates stratified by Universal POS (UPOS) tag.

Table 2: Acceptance rate by UPOS tag. Functional categories achieve near-perfect acceptance, while content-bearing categories are substantially harder.

UPOS Tag	Acceptance (α)	Count
ADP	1.0000	8
AUX	1.0000	3
PRON	0.7500	4
PUNCT	0.5783	83
NOUN	0.5333	15
PROPN	0.2857	21
VERB	0.0833	12

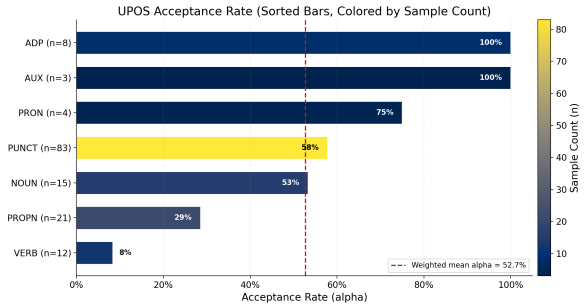

Figure 11: UPOS-stratified acceptance rates. ADP and AUX are perfectly predicted, while VERB drops to 8.3%—reflecting Hindi's rich verbal morphology (करता/करती/करते).

Method choice. For morphological analysis, we use a *fixed-depth TrueExit-style* setup at depth $d = 22$ (with standard exit heads) rather than adaptive policies. This is a controlled design choice: by keeping both policy and depth fixed, UPOS-wise acceptance differences can be attributed primarily to linguistic difficulty, not to exploration dynamics or depth-switching behavior. Depth 22 was selected because it is the strongest fixed-depth operating point in our depth analysis, so the resulting category-level acceptance trends are measured in the most reliable fixed-depth regime.

Interpretation.

- **ADP (1.00) and AUX (1.00)**: Postpositions and auxiliaries in Hindi are drawn

from small, closed-class sets and are highly predictable from syntactic context. Exit heads capture these distributions perfectly.

- **PRON (0.75)**: Pronouns are a relatively small class but carry gender/number inflection, introducing modest ambiguity.
- **PUNCT (0.58)**: Punctuation acceptance is moderate, likely because punctuation placement in Hindi is less rigidly determined than in English.
- **NOUN (0.53)**: Nouns span a large vocabulary and carry case marking, reducing predictability.
- **PROPN (0.29)**: Proper nouns are inherently difficult—they are open-class and context-dependent.
- **VERB (0.08)**: Verbs are the hardest category, reflecting Hindi's rich verbal morphology with agreement in person, number, and gender.

Caveat: Sample sizes for ADP (8), AUX (3), and PRON (4) are small, so their perfect or near-perfect acceptance should be interpreted with caution. PUNCT (83) and PROPN (21) have more reliable sample sizes.

5.8 Hook vs. Bottleneck Training Diagnostics

Figures 12–14 compare the training dynamics of Hook and Bottleneck head architectures across three dimensions: training loss, validation loss, and acceptance rate during training.

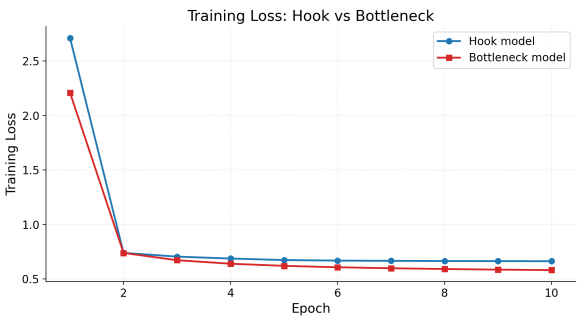

Figure 12: Training loss. The Bottleneck head converges to a lower loss than the Hook head. The gap widens after epoch 3, suggesting the bottleneck's compressed representation provides a more effective learning signal.

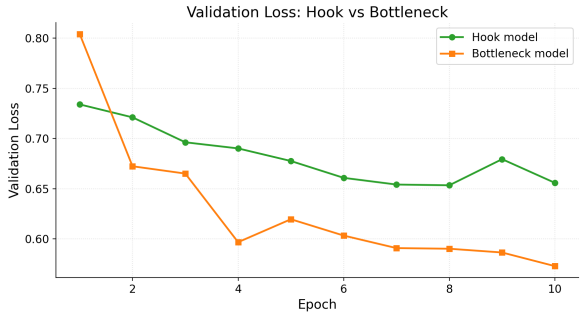

Figure 13: Validation loss. Both heads show no significant overfitting (validation closely tracks training). The Bottleneck head maintains its advantage, confirming genuine generalisation.

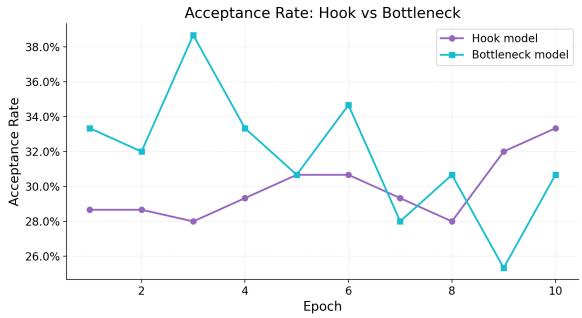

Figure 14: Hook vs. Bottleneck acceptance rate during training. The Bottleneck head achieves higher α throughout and continues to improve at epoch 10, while the Hook head plateaus earlier.

Interpretation. The training diagnostics provide a mechanistic explanation for the inference-time results:

1. The bottleneck's dimensionality reduction acts as a regulariser, forcing the head to learn a more compressed and generalisable representation of the target distribution.
2. Both architectures avoid overfitting (validation tracks training closely), so the Bottleneck's advantage is not due to capacity matching but to representation quality.

6 Discussion

6.1 Comparative Analysis of Trade-offs

Our 12-method evaluation reveals a three-dimensional trade-off space:

Acceptance vs. Speedup. No method simultaneously maximises both. BN-TrueExit achieves the highest acceptance (70.9%) at a

speedup of 0.796×, while Thomp-BN achieves the highest speedup (1.130×) at a moderate acceptance of 54.8%. The gap between these two reflects a fundamental tension: higher acceptance requires deeper exits or more conservative policies, which cost more computation per draft token.

Speedup vs. Throughput. Speedup and throughput are closely correlated but not identical, as throughput also accounts for tokens generated per step (not just wall-clock ratio). Thomp-BN leads on both metrics (1.130×, 30.40 Tok/s), confirming that its adaptive depth selection produces more tokens per unit time, not just faster individual steps.

BN vs. Hook. Across all policy families, the Bottleneck variant consistently outperforms the Hook variant:

- Thomp-BN (1.130×) vs. Thompson (1.054×): +7.2% speedup.
- UCB-BN (0.919×) vs. UCB-Hook (0.578×): +59.0% speedup.
- WThomp-BN (0.876×) vs. WThomp-Hook (0.574×): +52.6% speedup.

This systematic advantage stems from the Bottleneck head's superior training dynamics (lower loss, higher α throughout training) as shown in §5.8.

6.2 Method Ranking Summary

- **Best acceptance**: BN-TrueExit (70.9%) — best choice when output fidelity is paramount.
- **Best speedup**: Thomp-BN (1.130×) — best choice when latency is the primary concern.
- **Best throughput**: Thomp-BN (30.40 Tok/s) — the only method exceeding the AR baseline.
- **Most robust**: Thomp-BN maintains > 1.0× speedup at 500-sample scale (1.041×).
- **Best overall**: Thomp-BN offers the strongest combined profile (speedup, throughput, and robustness), though BN-TrueExit is preferred for quality-critical applications.

6.3 Deployment Recommendations

Based on our findings, we offer the following practical guidance:

1. **Latency-sensitive applications** (e.g., real-time translation, chatbots): Deploy **Thomp-BN**. It is the only method that consistently delivers wall-clock gains over AR decoding, with 14.4% higher throughput and robust performance at scale.
2. **Quality-critical applications** (e.g., document generation, formal text): Deploy **BN-TrueExit**. Its 70.9% acceptance rate means fewer rejected drafts and higher fidelity to the full model's distribution. The sub-unity speedup (0.796×) is acceptable when quality is non-negotiable.
3. **Balanced deployment**: **UCB-BN** or **WThomp-BN** offer a middle ground — acceptance rates above 60% with speedups of 0.87–0.92×, approaching but not exceeding the AR baseline.
4. **Use $K = 3$**: The K-ablation confirms this as the optimal draft length for our acceptance-rate regime.

7 Limitations

1. **Single language**: All experiments are conducted on Hindi. While Hindi's rich morphology makes it a challenging testbed, results may not directly transfer to other morphologically rich languages (e.g., Turkish, Finnish) or to morphologically simpler languages (e.g., Mandarin).
2. **Single model scale**: We evaluate only Qwen2-1.5B. The original EESD paper (Liu et al., 2024) reports higher acceptance rates on LLaMA-2-7B, suggesting that larger models may have more informative intermediate representations. Scaling behaviour remains unexplored.
3. **True early exit not universally implemented**: While TrueExit and BN-TrueExit demonstrate the benefit of stopping at the exit layer, the adaptive policies (Thompson, UCB) still run the full model during drafting. Combining adaptive policies with true early exit would likely yield further speedups.

8 Conclusion

We have presented a comprehensive evaluation of Early-Exit Speculative Decoding for Hindi text generation, comparing 12 decoding strategies across acceptance rate, speedup, throughput, morphological behaviour, and training dynamics.

Our key findings are:

1. **BN-TrueExit** achieves the highest acceptance rate (70.9%), demonstrating that bottleneck exit heads with true early exit best approximate the full model's output distribution.

2. **Thomp-BN** delivers the best wall-clock speedup (1.130×) and throughput (30.40 Tok/s), the only method to exceed the autoregressive baseline. Its adaptive Thompson Sampling policy with bottleneck heads provides the strongest combined profile of speed, throughput, and robustness.

3. The **Bottleneck head architecture** consistently outperforms the standard Hook head across all policy families, both in training dynamics (lower loss, higher α) and inference metrics (higher speedup, higher throughput).

4. **Morphological analysis** reveals a clear hierarchy: functional categories (ADP, AUX) are near-perfectly predicted by exit heads, while content-bearing categories with rich inflectional morphology (VERB at 8.3%, PROPN at 28.6%) remain challenging.

5. Results are **robust at scale**: the 500-sample evaluation preserves method rankings, with Thomp-BN sustaining above-unity speedup (1.041×).

These results establish Early-Exit Speculative Decoding as a viable acceleration strategy for morphologically rich languages, with clear trade-offs that can be navigated based on deployment requirements. The combination of adaptive policies with bottleneck head architectures emerges as the most promising direction for practical deployment.

References

Sangmin Bae, Jongwoo Ko, Hwanjun Song, and Se-Young Yun. 2023. Fast and robust early-exiting framework for autoregressive language models with synchronized parallel decoding. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. https://aclanthology.org/2023.emnlp-main.362.

Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2023. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*. https://arxiv.org/abs/2401.10774.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*. https://arxiv.org/abs/2302.01318.

Sumanth Doddapaneni, Gowtham Ramesh, Mitesh M. Khapra, Anoop Kunchukuttan, and Pratyush Kumar. 2021. A primer on pretrained multilingual language models. *arXiv preprint arXiv:2107.00676*. https://arxiv.org/abs/2107.00676.

Tahmid Hasan, Abhik Bhattacharjee, Md. Saiful Islam, Kazi Mubasshir, Yuan-Fang Li, Yong-Bin Kang, M. Sohel Rahman, and Rifat Shahriyar. 2021. XL-Sum: Large-scale multilingual abstractive summarization for 44 languages. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 4693–4703. https://aclanthology.org/2021.findings-acl.413.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*. https://arxiv.org/abs/1503.02531.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*, pages 19274–19286. https://proceedings.mlr.press/v202/leviathan23a.html.

Jiahao Liu, Qifan Wang, Jingang Wang, and Xunliang Cai. 2024. Speculative decoding via early-exiting for faster LLM inference with Thompson sampling control mechanism. In *Findings of the Association for Computational Linguistics: ACL 2024*. https://arxiv.org/abs/2406.03853.

Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Q. Tran, Yi Tay, and Donald Metzler. 2022. Confident adaptive language modeling. In *Advances in Neural Information Processing Systems (NeurIPS 2022)*. https://arxiv.org/abs/2207.07061.

Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. 2018. Blockwise parallel decoding for deep autoregressive models. In *Advances in Neural Information Processing Systems (NeurIPS 2018)*. https://arxiv.org/abs/1811.03115.

Surat Teerapittayanon, Bradley McDanel, and H.T. Kung. 2016. BranchyNet: Fast inference via early exiting from deep neural networks. In *Proceedings of the 23rd International Conference on Pattern Recognition (ICPR 2016)*, pages 2464–2469. https://doi.org/10.1109/ICPR.2016.7900006.

Heming Xia, Zhe Yang, Qingxiu Dong, Peiyi Wang, Yongqi Li, Tao Ge, Tianyu Liu, Wenjie Li, and Zhifang Sui. 2024. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. In *Findings of the Association for Computational Linguistics: ACL 2024*. https://arxiv.org/abs/2401.07851.

Ji Xin, Raphael Tang, Jaejun Lee, Yaoliang Yu, and Jimmy Lin. 2020. DeeBERT: Dynamic early exiting for accelerating BERT inference. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL 2020)*, pages 2246–2251. https://aclanthology.org/2020.acl-main.204.

An Yang, Baosong Yang, Binyuan Hui, and others. 2024. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*. https://arxiv.org/abs/2407.10671.

Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, and Sharad Mehrotra. 2023. Draft & verify: Lossless large language model acceleration via self-speculative decoding. *arXiv preprint arXiv:2309.08168*. https://arxiv.org/abs/2309.08168.